

Carrera:
Programación y WebMaster

Docente:
MTRO. Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Salomon Campos Pablo Leonel

Fecha:
12 de Junio 2026

Introducción

El desarrollo de software es importante elegir una buena arquitectura, ya que esta define la forma en la que se organiza y funciona un sistema. Los patrones de arquitectura ayudan a ordenar el código, separar responsabilidades y facilitar el mantenimiento de las aplicaciones.

Parte 1: Investigación

Investiga cada uno de los patrones solicitados.

Aspecto	MVC (Model-View-Controller)	Arquitectura en Capas (Layered Architecture)	Microservicios	Monolítica	Cliente-Servidor
Definición	Patrón que divide una aplicación en modelo, vista y controlador. Su propósito es separar los datos, la interfaz y el manejo de las acciones del usuario.	Forma de organizar un sistema en niveles con responsabilidades diferentes. Cada capa atiende una parte del proceso para mantener el software ordenado.	Arquitectura que separa una aplicación en servicios pequeños e independientes. Cada servicio cumple una función concreta del sistema.	Modelo donde toda la aplicación se construye como una sola unidad. Las funciones principales se encuentran dentro de un mismo proyecto.	Modelo donde un cliente solicita información o servicios y un servidor responde. Es común en aplicaciones que trabajan mediante una red.
Estructura general	Se compone de tres secciones relacionadas: el modelo gestiona información, la vista la presenta y el controlador coordina las solicitudes.	Se organiza en capas como presentación, lógica de negocio, acceso a datos y almacenamiento. La información pasa de una capa a otra.	El sistema se divide en servicios separados que se comunican por APIs o mensajería. Puede incluir un punto de entrada para coordinar peticiones.	Tiene una estructura centralizada. Aunque puede contar con módulos internos, todos forman parte de la misma aplicación y se despliegan juntos.	Se estructura con clientes, servidor, red de comunicación y, generalmente, una base de datos administrada desde el servidor.
Componentes principales	Modelo, vista y controlador.	Capa de presentación, capa de negocio, capa de acceso a datos y base de datos.	Servicios independientes, APIs, base de datos por servicio, gateway y mecanismos de comunicación.	Interfaz de usuario, lógica de negocio, acceso a datos, módulos internos y base de datos.	Cliente, servidor, red, protocolos de comunicación y base de datos.
Forma de funcionamiento	El usuario interactúa con la vista. El controlador recibe la acción, consulta o modifica el modelo y devuelve una respuesta mediante la vista.	La solicitud inicia en la presentación, pasa a la lógica de negocio, consulta datos si es necesario y regresa una respuesta al usuario.	Una petición se dirige al servicio correspondiente. Si se requiere, ese servicio se comunica con otros para completar la operación.	La aplicación recibe la solicitud y la procesa dentro del mismo sistema. Todas las funciones trabajan desde un solo conjunto de código.	El cliente envía una petición al servidor. El servidor procesa la información, consulta los datos necesarios y envía una respuesta al cliente.
Ventajas	Facilita el orden del código, separa responsabilidades y permite mantener o modificar partes del sistema con mayor claridad.	Mejora la organización, facilita el mantenimiento y ayuda a ubicar mejor la lógica de cada parte del sistema.	Permite escalar servicios de forma individual, facilita el trabajo por equipos y reduce el impacto de fallas aisladas.	Es más sencilla de desarrollar al inicio, requiere menos infraestructura y resulta práctica para proyectos pequeños o prototipos.	Centraliza la información, permite acceso desde varios dispositivos y facilita la administración de datos desde el servidor.
Desventajas	Puede ser innecesario en proyectos muy simples. Si no se controla, el controlador puede acumular demasiadas funciones.	Puede volverse rígida si el sistema necesita cambios constantes. También puede afectar el rendimiento si hay demasiados niveles.	Aumenta la complejidad técnica porque requiere controlar comunicación, seguridad, despliegue y monitoreo de varios servicios.	Cuando crece demasiado, puede ser difícil de mantener o escalar. Un cambio pequeño puede afectar otras partes de la aplicación.	Depende de la conexión y del funcionamiento del servidor. Si el servidor falla, los clientes pueden perder el acceso.
Casos de uso	Aplicaciones web, sistemas escolares, plataformas administrativas y proyectos desarrollados con frameworks como Laravel.	Sistemas empresariales, bancarios, escolares, inventarios y aplicaciones donde se necesita separar claramente las responsabilidades.	Plataformas grandes, comercio electrónico, streaming, banca digital y sistemas con muchos usuarios o módulos independientes.	Sistemas internos, proyectos escolares, aplicaciones pequeñas, prototipos y soluciones que no requieren crecimiento complejo.	Páginas web, aplicaciones móviles, correos electrónicos, sistemas escolares, bancos en línea y videojuegos conectados a servidores.

Parte 2: Cuadro comparativo

Elabora un cuadro comparativo considerando los siguientes criterios:

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar datos, pantallas y acciones para que el sistema quede más ordenado.	Dividir el sistema por niveles, de acuerdo con la función de cada parte.	Reunir la aplicación en un solo proyecto que trabaje como una unidad.	Separar el sistema en servicios pequeños que funcionen de forma independiente.	Permitir que un cliente solicite información y que un servidor la procese.
Estructura	Modelo, vista y controlador.	Presentación, lógica de negocio, acceso a datos y base de datos.	Un solo sistema con módulos internos conectados entre sí.	Varios servicios, APIs, bases de datos separadas y, en algunos casos, un gateway.	Cliente, servidor, red de comunicación y base de datos.
Organización del código	El código se reparte según su responsabilidad: datos, interfaz o control.	Se ordena por capas para evitar mezclar la interfaz con las reglas del sistema.	Todo queda dentro del mismo proyecto; puede organizarse por carpetas o módulos.	Cada servicio tiene su propio código y atiende una función concreta.	El cliente maneja la interacción y el servidor conserva la lógica principal.
Escalabilidad	Puede escalar de forma moderada si el proyecto está bien organizado.	Escala bien en sistemas medianos, aunque puede volverse pesada en proyectos muy grandes.	Es limitada, porque normalmente se escala toda la aplicación completa.	Es alta, ya que se puede escalar solo el servicio que lo necesite.	Depende de la capacidad del servidor y de la cantidad de clientes conectados.
Mantenimiento	Es manejable porque cada parte tiene una función clara.	Suele ser sencillo, pues los cambios se ubican en la capa correspondiente.	Al principio es fácil, pero con el crecimiento puede complicarse bastante.	Permite modificar servicios por separado, aunque exige más control técnico.	Es práctico si el servidor está bien organizado y documentado.
Complejidad	Media; no es difícil, pero requiere respetar la separación de partes.	Media; se entiende bien, aunque puede tener varias capas.	Baja al inicio, porque todo se trabaja en un mismo lugar.	Alta; necesita comunicación entre servicios, despliegues y monitoreo.	Media; la idea es simple, pero depende de red, servidor y seguridad.
Facilidad de implementación	Es relativamente sencilla usando frameworks web como Laravel o similares.	Fácil de aplicar en sistemas administrativos o empresariales.	Muy fácil para iniciar un proyecto pequeño o un prototipo.	Más difícil, porque requiere más configuración e infraestructura.	Fácil de entender y muy común en aplicaciones web y móviles.

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Rendimiento	Bueno si no se sobrecargan los controladores ni las consultas.	Puede ser estable, aunque muchas capas pueden hacer más lento el flujo.	Puede ser rápido al inicio porque todo está junto.	Puede rendir muy bien, pero depende de la comunicación entre servicios.	Depende mucho del servidor, la red y la cantidad de solicitudes.
Reutilización	Permite reutilizar modelos, vistas o controladores en partes similares.	Favorece reutilizar funciones de una capa en distintas partes del sistema.	La reutilización es posible, pero puede quedar muy ligada al mismo proyecto.	Alta, porque algunos servicios pueden usarse por diferentes aplicaciones.	El servidor puede ofrecer servicios reutilizables para varios clientes.
Casos de uso recomendados	Aplicaciones web, sistemas escolares, tiendas en línea y paneles administrativos.	Sistemas empresariales, inventarios, bancos, plataformas escolares y sistemas de gestión.	Proyectos pequeños, sistemas internos, prototipos y aplicaciones con poco crecimiento esperado.	Plataformas grandes, comercio electrónico, bancos digitales, streaming y apps con muchos usuarios.	Páginas web, aplicaciones móviles, correo electrónico, plataformas escolares y sistemas en red.
Ventajas	Ordena el desarrollo, facilita cambios y ayuda a separar responsabilidades.	Mejora la organización, el mantenimiento y la claridad del sistema.	Es simple, rápida de iniciar y no requiere tanta infraestructura.	Escala mejor, permite trabajo por equipos y reduce el impacto de algunas fallas.	Centraliza información, permite acceso desde distintos dispositivos y facilita administrar datos.
Desventajas	Puede generar muchos archivos y volverse confuso si no se organiza bien.	Puede ser rígida y perder rendimiento si se agregan capas innecesarias.	Cuando crece, es más difícil de modificar, probar y escalar.	Es más costosa y compleja de manejar; requiere monitoreo y buena infraestructura.	Si falla el servidor o la red, los usuarios pueden quedarse sin acceso.

Parte 3: Análisis

Después del cuadro comparativo, responde las siguientes preguntas.

- 1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?**
El mejor patrón a usar sería el MVC, por que este patrón permite separar el sistema en partes entendibles, donde va la información y reglas del negocio van en el modelo, la parte visual en la vista y las acciones del usuario en el controlador lo cual esto ayuda a que el proyecto no se vuelva muy desordenado desde el inicio.
- 2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?**
Los microservicios tienen la ventaja de dividir el sistema en servicios más pequeños, donde cada uno atiende una función en específico, también nos facilita la escalabilidad, porque si una función tiene más uso, se puede cambiar solo esa parte en específico y en comparación con una arquitectura monolítica, esta solo nos dan más flexibilidad, aunque también requieren más organización y conocimientos técnicos para administrarlos bien.
- 3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?**
El patrón MVC sigue siendo muy utilizado porque este se adapta bien a los funcionamientos de las aplicaciones web, ese tipo de sistemas normalmente hay muchos datos, pantallas y acciones de parte del usuario, y el patrón MVC permite ordenar todas esas partes de manera clara y organizada. Además que es usado por muchos frameworks que usan ese tipo de patrón.
- 4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?**
Unos de los principales riesgos es que el equipo de desarrollo termine invirtiendo más tiempo en entender la arquitectura que en desarrollar el sistema, también que en un proyecto pequeño, una arquitectura demasiado compleja puede provocar retrasos, errores innecesarios y dificultad para dar mantenimiento al sistema.
- 5. ¿Crees que existe un patrón mejor que todos los demás?**
No existe un patrón que sea el mejor, ya que cada patrón tiene sus ventajas y limitaciones, y su utilidad depende mucho del tipo de proyecto que se quiere desarrollar, al igual lo más importante es analizar las necesidades del sistema, el equipo de trabajo, el tiempo disponible y el crecimiento que podría tener el proyecto en el futuro y entender el flujo de trabajo.

Parte 4: Caso práctico.

Selecciona uno de los siguientes sistemas:

- Sistema escolar
- Patrón a usar: MVC (Modelo-Vista-Controlador)

1. Decidí elegir el sistema escolar, ya que es una plataforma muy común en instituciones educativas y puede incluir funciones como consulta de horarios, registro de calificaciones, control de asistencia, avisos escolares y administración de alumnos o docentes etc etc.

El patrón que utilizaría sería el MVC, porque este nos permite organizar el sistema en tres partes principales: el modelo, la vista y el controlador, lo cual nos ayuda a separar los datos, la interfaz y la lógica del sistema sin volver el proyecto demasiado atrás.

El número de usuarios, en el sistema escolar puede ser utilizado por alumnos, maestros, administrativos y directivos. Aunque puede tener varios usuarios y normalmente no requiere alguna arquitectura tan pesada como algún microservicio.

Respecto a la escalabilidad, el patron MVC nos permite que el sistema crezca por etapas. Primero se podrían desarrollar módulos básicos como usuarios, horarios y calificaciones, y después agregar reportes o notificaciones.

Sobre la complejidad, el patron MVC es adecuado porque mantiene una estructura clara y fácil de entender, para un proyecto escolar, usar una arquitectura demasiado compleja podría hacer que el desarrollo sea más lento.

El mantenimiento, el patron MVC también ayuda bastante, ya que cada parte del sistema queda separada, si se necesita cambiar una pantalla, se modifica la vista.

En los costos, el patrón MVC no requiere una infraestructura pesada, se puede trabajar con un solo proyecto y una base de datos principal, lo cual reduce gastos y facilita el desarrollo del proyecto.

Conclusión:

Con esta actividad pude comprender mejor que los patrones de arquitectura son importantes porque ayudan a organizar un sistema desde el inicio. No solo sirve para trabajar mejor el código, sino también para definir cómo se van a comunicar las partes del software.

El patrón que me pareció más interesante fue el MVC, porque es bastante usado en aplicaciones web y permite separar de forma clara la información, la interfaz y la lógica del sistema. Además, considero que es una arquitectura práctica para proyectos escolares o sistemas desarrollados por equipos pequeños, ya que no es tan difícil de entender.

Igual aprendí que la arquitectura influye mucho en la calidad del sistema. Si se elige una arquitectura adecuada, el sistema puede ser más fácil de mantener, más ordenado, más escalable y menos propenso a errores. En cambio, si se selecciona una arquitectura incorrecta, el proyecto puede volverse difícil de modificar y más costoso de mejorar.

Por eso es importante elegir bien la arquitectura desde el inicio del proyecto, escoger una buen patron ayuda a que desde las primeras etapas nos permita ahorrar tiempo, evitar problemas y facilitar el trabajo del equipo durante el desarrollo.

En conclusión, no existe un patrón perfecto para todos los casos, depende del tipo de sistema y las necesidades que este requiera.